MERGE SORT VS HEAP SORT

By: Gizelle De La Torre, Lifu Erhart, Mohamed Eljack, Henry Owen

```python
def heapify(arr, n, i):
    # Assume root is largest
    largest = i
    l = 2 * i + 1 # left child index
    r = 2 * i + 2 # right child index

    # check if left child exists and is > root
    if l < n and arr[l] > arr[largest]:
        largest = l
    # check if right child exists and is > largest
    if r < n and arr[r] > arr[largest]:
        largest = r
    # if largest is not the root, swap and heapify
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i
        heapify(arr, n, largest)
```

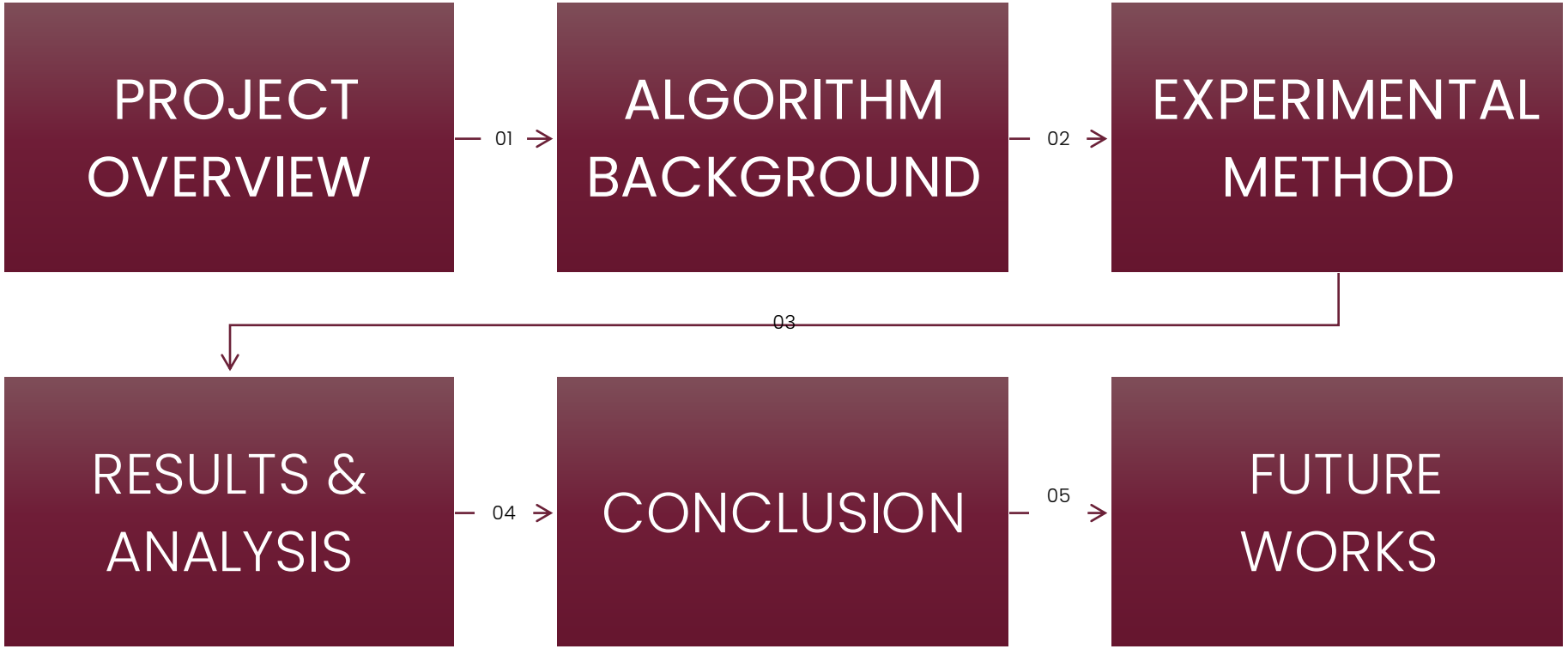
```python
def heapSort(arr):
    n = len(arr)

    if n <= 1:
        return

    # build max heap from array
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # move max to end, re-heapify
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)
```

AGENDA


PROJECT OVERVIEW
01
ALGORITHM BACKGROUND
02
EXPERIMENTAL METHOD
03
RESULTS & ANALYSIS
04
CONCLUSION
05
FUTURE WORKS

PROJECT OVERVIEW

Our project involves taking the empirical route where we take two types of sorting algorithms and compare their performances.

The two algorithms tested will be merge sort and heap sort, but they will be tested using the same problem. We will analyze, compare, and find their runtimes. The tests will be based off how long each algorithm takes to sort data of varied sizes.

This will help us figure out and conclude which algorithm has a better performance and the reasoning on why.

Another **algorithm will** be tested at the end using **a variation from heapsort** and **this** one is called double-pop heapsort.

```
mid = (left + right) // 2 # find midpoint
mergeSort(arr, left, mid) # recurse on left half
mergeSort(arr, mid + 1, right) # recurse of right half
```

ALGORITHM BACKGROUND

Heap Sort

- Heap sort is based on the algorithm Binary Heap.
- Builds max or min heap by using heapify.
- The algorithm works by going through the array and looking for the maximum, or sometimes minimum, element and committing a swap with either the first or last element.
- Heap sort has a time complexity of O(n log n).

Merge Sort

- A divide-and-conquer algorithm.
- The array is recursively split into halves and to become subproblems.
- Subproblem are solved and then merged back together to get a sorted array.
- Merge sort has a time complexity of O(n log n).

Double-pop Heapsort

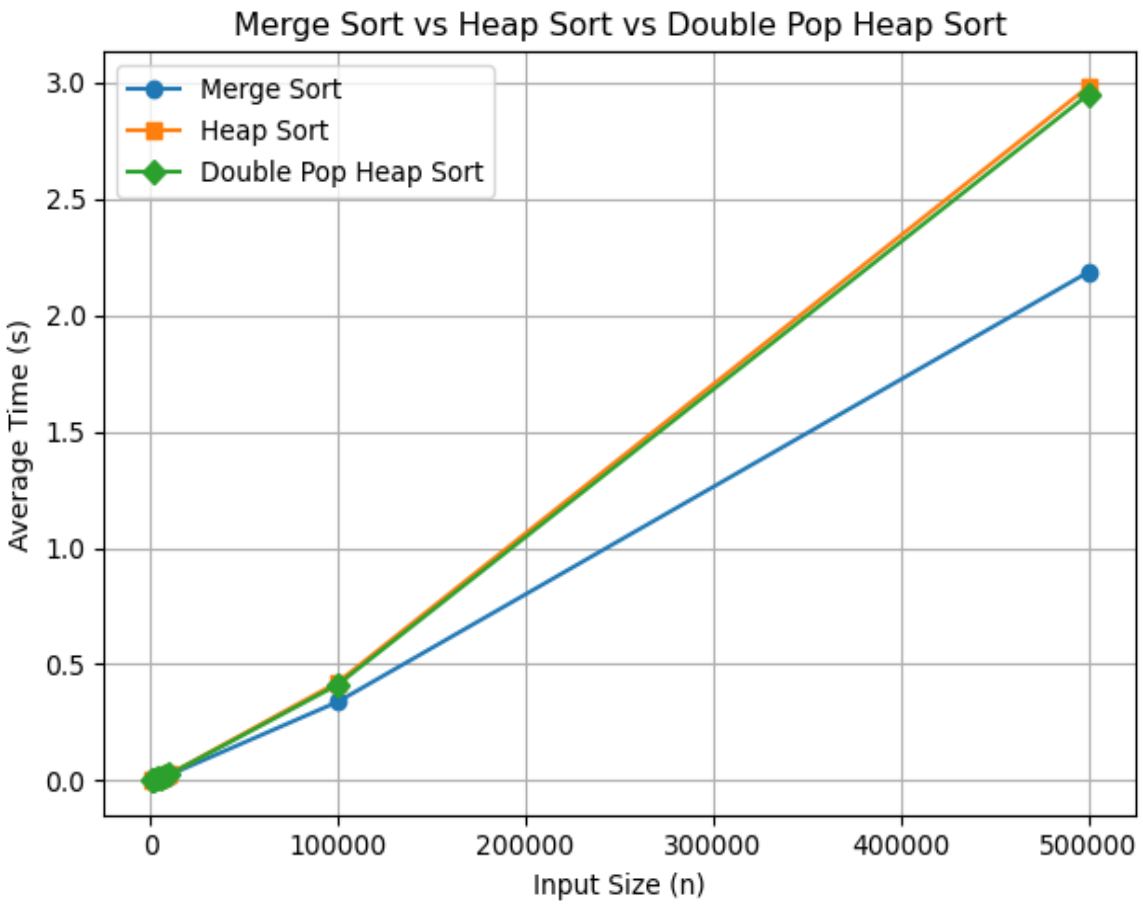
- A variation from the Heapsort algorithm.
- Extracting two largest elements each iteration (root node & one of the children).
- Swaps root with last element and then performs a re-heapify.
- Repeats process until the array is sorted.
- Double-pop Heapsort still maintains a time complexity O(n log n).

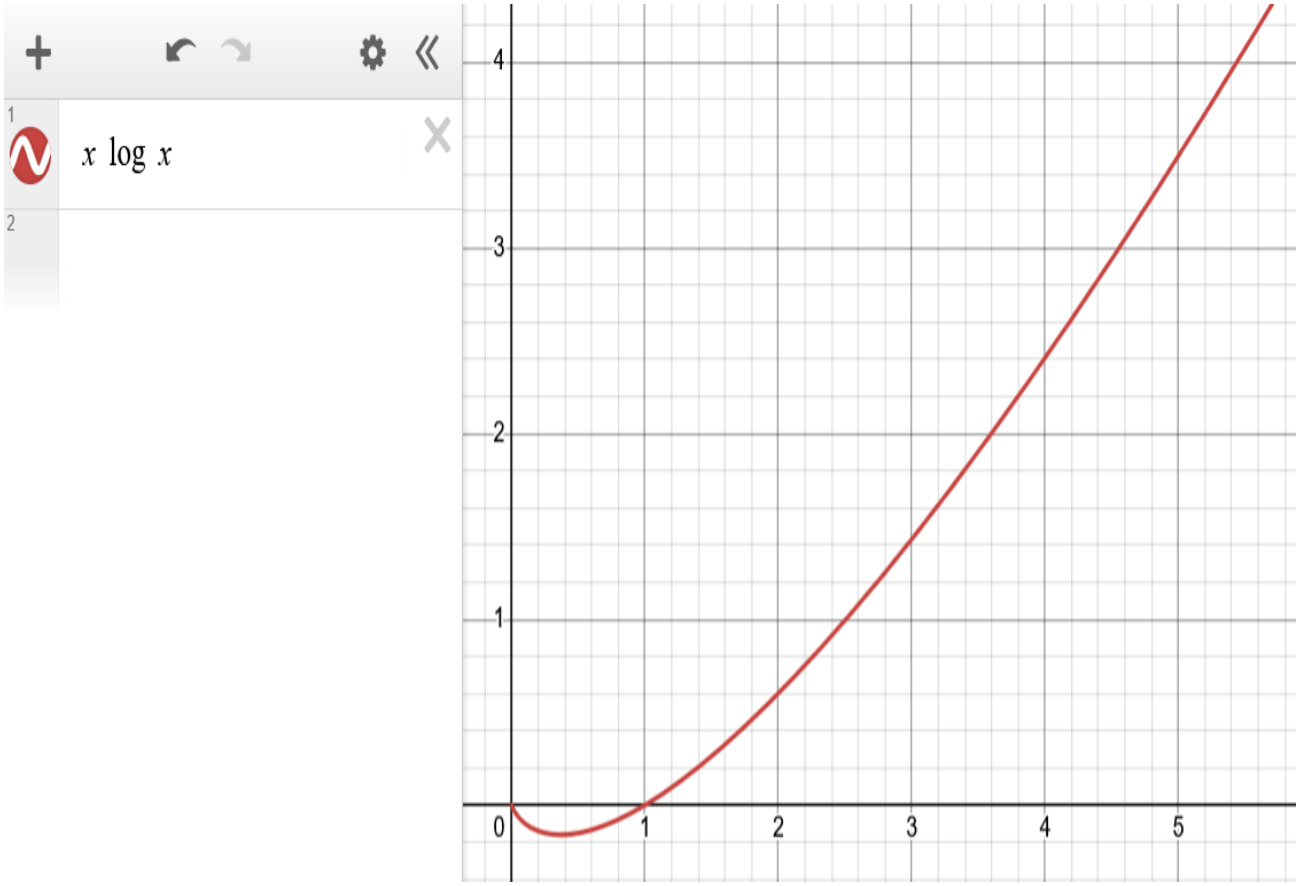
EXPERIMENTAL METHOD

We generated random arrays of different input sizes and ran both Merge Sort and Heap Sort on the exact same data to keep the test fair. Each test was repeated multiple times, and we recorded the average runtime to reduce any random variation.

After that, we ran the Double Pop Heap Sort using the same process and compared all three algorithms by graphing their runtimes as the input size increased.

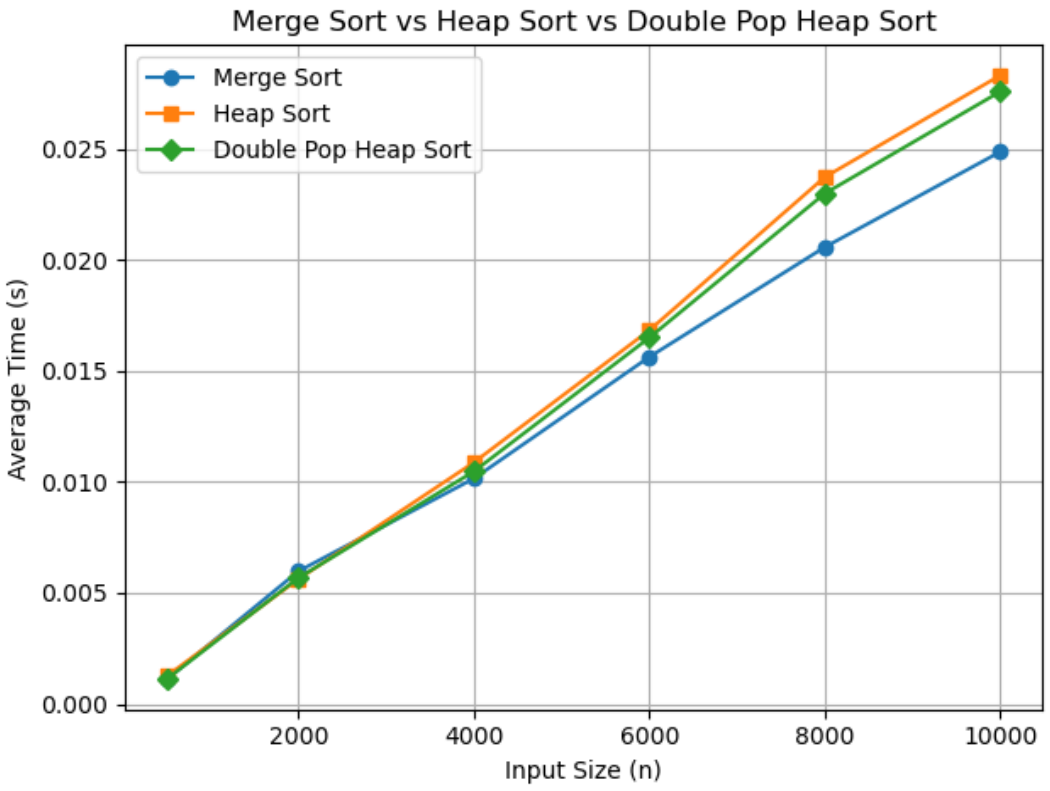
RESULTS RELATE TO THE EXPECTED N LOG N TIME COMPLEXITY

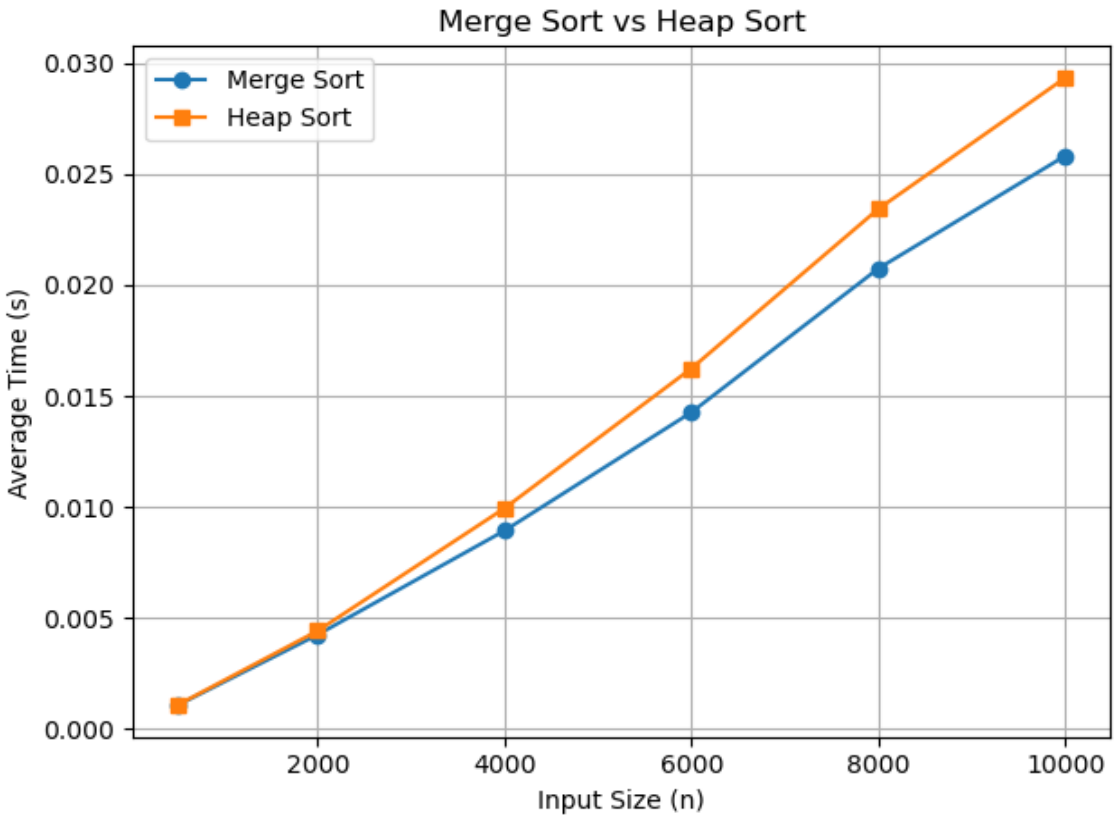




Our data follows the expected pattern, with runtime increasing faster than linear but slower than n squared, which confirms that all three algorithms behave as expected.

MERGE SORT VS HEAP SORT





MERGE SORT V.S HEAP SORT RESULTS

- As the input size increases, the runtime increases for all three algorithms which is expected
- The lines grow at a similar rate showing they all follow n log n time complexity
- Merge Sort is always the lowest on the graph, so it is the fastest
- Heap Sort is higher at every input size, so it takes more time
- Double Pop Heap Sort is slightly faster than regular Heap Sort but still slower than Merge Sort
- **Merge Sort** is **faster** because it does **simpler operations,** while Heap Sort has extra work maintaining the heap

Overall, all three algorithms are efficient, but Merge Sort performs the best while the double pop method only slightly improves Heap Sort

CONCLUSION

Even though all three algorithms have the same time complexity of n log n, they do not perform the same in practice. The results showed that Merge Sort was consistently the fastest, while Heap Sort took more time.

The Double Pop Heap Sort slightly improved performance compared to regular Heap Sort because removing two elements at once reduces the number of heap operations.

However, it still did not outperform Merge Sort. This shows that real world performance depends not just on complexity, but also on how the algorithm actually works and is implemented.

COMPARING PERFORMANCE ON DIFFERENT INPUT TYPES

Testing different input types helps answer whether an algorithm's performance changes depending on the data it is given.

By using sorted, reverse sorted, and duplicate heavy arrays, we can see how each algorithm behaves under different conditions and identify potential best- and worst-case scenarios.

This adds to our project by giving a more complete understanding of how these algorithms perform beyond just average runtime.

Examples:

- Test algorithms on different input data
- Already sorted arrays
- Reverse sorted arrays
- Arrays with many duplicate values
- Compare how performance changes across cases
- Shows behavior beyond just random data

COMPARING MEMORY EFFICIENCY

Testing memory efficiency helps us understand whether an algorithm's performance comes at the cost of using more space.

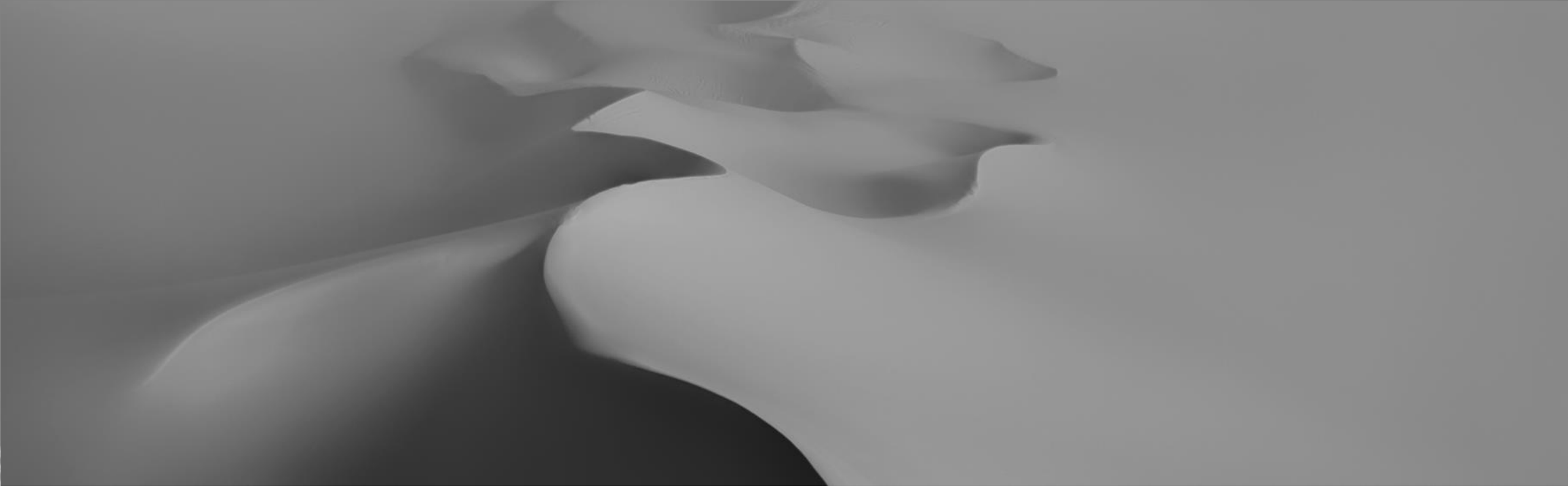
In our original tests, we only focused on runtime, which does not give the full picture of how efficient an algorithm really is.

How We Test it:

By analyzing how much memory each algorithm uses during execution, we can compare their space efficiency and identify tradeoffs between speed and memory usage.

Why it Matters:

This would give us a more complete understanding of how these algorithms perform beyond just runtime.



THANK YOU